

Data de Execução: 02/03/2025

Semana 4 Aula 01 Introdução às APIs

Cabeçalho de Identificação:

- **Semana:** 4 - **Aula:** 1
- **Tópico Principal:** APIs e serviços web
- **Subtítulo/Tema Específico:** Introdução às APIs
- **Código da Aula:** [SIS]ANO2-C2-B1-S4-A1
- **Objetivos da Aula:** 1. Conhecer os conceitos básicos de APIs. 2. Compreender a importância das APIs na integração de sistemas modernos.
- **Recursos Adicionais:** Recurso audiovisual (vídeos/imagens), lápis e caderno.

Exposição Detalhada do Conteúdo

[Slide 11 - Construindo conceito: O que são APIs]

- **Definição:** APIs (*Application Programming Interfaces*) são intermediárias que oferecem regras e definições para que componentes de software interajam entre si. Elas permitem a integração e comunicação entre sistemas distintos.
- **Aprofundamento (Busca Externa):** Uma API funciona como um "contrato" entre um provedor e um consumidor. No desenvolvimento web moderno, as APIs permitem que o *Front-end* (interface) converse com o *Back-end* (servidor/banco de dados) de forma padronizada.
- **Exemplo Prático:** Imagine um site de e-commerce que precisa calcular o frete. Ele utiliza uma API dos Correios para enviar o CEP do cliente e receber o valor e o prazo de entrega em tempo real, sem precisar ter todo o banco de dados dos Correios instalado em seu próprio servidor.

[Slide 13 - Construindo conceito: Definição e Função]

- **Definição:** Uma API define métodos e padrões de comunicação, permitindo que um aplicativo solicite dados ou funções de outro sem entender sua complexidade interna. Utilizam *endpoints* (URLs específicas) para operações CRUD.
- **Aprofundamento (Busca Externa):** O termo "abstração" é fundamental aqui: o desenvolvedor não precisa saber como o algoritmo de busca do Google Maps funciona por dentro, apenas como "chamar" a API para obter as coordenadas. [Informação complementar via busca externa].

[Slide 17 - Construindo conceito: Tipos de APIs]

- **Definição:** Existem diferentes estilos:
 1. **REST:** O mais comum, usa HTTP e URLs para identificar recursos.
 2. **SOAP:** Protocolo baseado em XML para transações seguras e complexas.
 3. **GraphQL:** Permite consultas flexíveis, onde o cliente pede exatamente o que precisa.
- **Aprofundamento (Busca Externa):** Enquanto o REST entrega um pacote fixo de dados (JSON), o GraphQL é como um self-service onde você escolhe os campos, evitando o *over-fetching* (receber dados demais). [Informação complementar via busca externa].

Fechamento da Aula

- **Resumo:** 1. APIs são "pontes" de comunicação entre softwares distintos. 2. Facilitam a integração, modularidade e reutilização de código. 3. REST, SOAP e GraphQL são os padrões predominantes no mercado.
- **Curadoria Digital:** [O que é uma API? | Dicionário do Programador](#) - Tutorial excelente para entender o conceito de forma visual.
- **Bibliografia:** 1. RICHARDSON, L.; RUBY, S. **RESTful Web Services**. Sebastopol: O'Reilly Media, 2007.
2. FIELDING, R. T. **Architectural Styles and the Design of Network-based Software Architectures**. Irvine: University of California, 2000.

Aula 02: Desenvolvimento de APIs com Node.js

Cabeçalho de Identificação:

- **Semana:** 4 - **Aula:** 2
- **Tópico Principal:** APIs e serviços web
- **Subtítulo/Tema Específico:** Desenvolvimento de APIs com Node.js
- **Código da Aula:** [SIS]ANO2-C2-B1-S4-A2
- **Objetivos da Aula:** 1. Desenvolver APIs utilizando o ambiente Node.js. 2. Compreender a arquitetura orientada a eventos e o ecossistema npm.
- **Recursos Adicionais:** Computador, VS Code, Node.js instalado.

Exposição Detalhada do Conteúdo

[Slide 7 - Construindo conceito: O que é Node.js]

- **Definição:** É um ambiente de execução JavaScript fora do navegador, usado para criar servidores e ferramentas de automação.
- **Aprofundamento (Busca Externa):** O Node.js é construído sobre o motor V8 do Google Chrome, o que garante alta velocidade de execução do código JavaScript no servidor. [Informação complementar via busca externa].

[Slide 8 - Construindo conceito: Vantagens do Node.js]

- **Definição:** Destaca-se pela arquitetura orientada a eventos e I/O não-bloqueante, permitindo gerenciar múltiplas requisições simultâneas eficientemente. Possui o ecossistema `npm` para pacotes prontos.
- **Aprofundamento (Busca Externa):** O modelo *Single-Threaded Event Loop* do Node permite que ele não "trave" enquanto espera uma resposta do banco de dados, podendo atender outros clientes nesse intervalo. [Informação complementar via busca externa].

[Slide 9 - Construindo conceito: Componentes Principais]

- **Definição:** O **Express.js** é o framework padrão para roteamento e middlewares. **Middlewares** são funções que interceptam a requisição (ex: para autenticação) antes dela chegar ao destino final.
- **Aprofundamento (Busca Externa):** O uso do Express reduz drasticamente a quantidade de código necessária para criar um servidor HTTP puro em Node. [Informação complementar via busca externa].
- **Exemplo Prático:** Em um sistema de restaurante, um middleware pode verificar se o usuário está logado antes de permitir que ele faça o pedido de uma pizza via API.

Fechamento da Aula

- **Resumo:** 1. Node.js permite usar JavaScript no Back-end com alta escalabilidade. 2. Express.js simplifica a criação de rotas e o tratamento de requisições HTTP. 3. Middlewares garantem segurança e processamento prévio de dados.
- **Curadoria Digital:** [Criando uma API com Node.js e Express | Guia do Programador](#) - Tutorial prático de "Hello World" em APIs.
- **Bibliografia:** 1. TEIXEIRA, P. **Professional Node.js: Building Comprehensive Developer Solutions**. Wrox, 2012.
2. CANTELON, M. et al. **Node.js in Action**. Manning Publications, 2014.

Aula 03: Desenvolvimento de APIs com Django

Cabeçalho de Identificação:

- **Semana:** 4 - **Aula:** 3
- **Tópico Principal:** APIs e serviços web
- **Subtítulo/Tema Específico:** Desenvolvimento de APIs com Django
- **Código da Aula:** [SIS]ANO2-C2-B1-S4-A3
- **Objetivos da Aula:** 1. Desenvolver APIs robustas utilizando o framework Django (Python). 2. Implementar operações CRUD completas com Django Rest Framework (DRF).
- **Recursos Adicionais:** Python instalado, VS Code, acesso à internet.

Exposição Detalhada do Conteúdo

[Slide 6 - Colocando em prática: Configuração do Django]

- **Definição:** O processo inicia com a criação do projeto e da aplicação (`startproject` e `startapp`). Define-se o **Modelo** (classe Python que representa a tabela no banco) e o **Serializer** (converte objetos Python em JSON).
- **Aprofundamento (Busca Externa):** O Django segue o princípio "Batteries Included" (pilhas incluídas), trazendo ferramentas prontas para autenticação, administração e ORM (Mapeamento Objeto-Relacional). [Informação complementar via busca externa].

[Slide 6 - Operações CRUD e Views]

- **Definição:** Utiliza-se o **Django Rest Framework (DRF)** para criar *views* baseadas em classes que lidam automaticamente com listagem, criação e exclusão de dados.
- **Exemplo Prático:** Criação de uma API para gerenciar um inventário de produtos, permitindo que uma loja adicione novos itens (POST), liste o estoque (GET) ou remova produtos esgotados (DELETE).

Fechamento da Aula

- **Resumo:** 1. Django é um framework Python focado em rapidez e segurança. 2. O DRF (Django Rest Framework) é a ferramenta essencial para transformar o Django em uma fábrica de APIs. 3. Serializers são a peça-chave para a tradução de dados entre o banco e o formato JSON.
- **Curadoria Digital:** [Criando uma API REST com Python e Django | Código Fonte TV](#) - Explicação didática sobre as vantagens do Django.
- **Bibliografia:** 1. PERCIVAL, H.; ELMAN, G. **Test-Driven Development with Python**. O'Reilly Media, 2017.
2. VINCENT, W. S. **Django for APIs: Build web APIs with Python and Django**. WelcomeToCode, 2022

